

ใบงานที่ 1 ปฏิบัติการแทนที่ข้อมูลในหน่วยความจำ

คำชี้แจง:

1. ให้นักศึกษาปฏิบัติการทดลองตามโจทย์ที่กำหนดให้
2. บันทึกขั้นตอนการทดลอง ปัญหา และการแก้ไขปัญหา
3. บันทึกผลลัพธ์การทดลองของแต่ละการทดลอง
4. ตอบคำถามท้ายการทดลอง
5. สรุปผลการทดลอง
6. จัดทำลงในไฟล์ Microsoft Word ในรูปแบบใบงาน
7. ส่งงานในเวลา (สถานที่ส่งงานตามผู้สอนกำหนดให้)

การทดลองที่ 1: ตรวจสอบขนาดข้อมูลและ pointer

ให้นักศึกษาเขียนโปรแกรมตรวจสอบขนาดของชนิดข้อมูลบนเครื่องตนเอง

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main(void) {
```

```
    printf("sizeof(char) = %zu\n", sizeof(char));
```

```
    printf("sizeof(int) = %zu\n", sizeof(int));
```

```
    printf("sizeof(float) = %zu\n", sizeof(float));
```

```
    printf("sizeof(double) = %zu\n", sizeof(double));
```

```
    printf("sizeof(bool) = %zu\n", sizeof(bool));
```

```
    printf("sizeof(int*) = %zu\n", sizeof(int*));
```

```
    printf("sizeof(char*) = %zu\n", sizeof(char*));
```

```
    return 0;
```

```
}
```

PS D:\ENGCE124\lab1> ./1-1.exe

sizeof(char) = 1

sizeof(int) = 4

sizeof(float) = 4

sizeof(double) = 8

sizeof(bool) = 1

sizeof(int*) = 8

sizeof(char*) = 8

คำถามท้ายการทดลอง:

1. เครื่องของนักศึกษาเป็น 32-bit หรือ 64-bit พิจารณาจากอะไร
พิจารณาจาก sizeof(int*) และ sizeof(char*) ที่เป็นตัวแปรพ้อยเตอร์มีขนาด 8-bytes ซึ่งเครื่องคอมพิวเตอร์ที่มีขนาด 8 bytes หมายถึงเครื่องคอมพิวเตอร์ที่เป็น 64-bit ส่วน 32-bit จะมีขนาด 4-bytes
2. sizeof(int*) และ sizeof(char*) เท่ากันหรือไม่ เพราะอะไร
มีขนาดเท่ากันเพราะมีขนาดที่ 8-bytes ตามผลลัพธ์ที่แสดง เพราะอ้างอิงจากการแสดงจำนวนพื้นที่ข้อมูล และพบว่าพื้นที่ข้อมูลเท่ากัน
3. sizeof(char) เท่ากับ 1 หมายถึง 1 ตัวอักษรเสมอหรือไม่
ใช่ เพราะหลักการเก็บตัวแปรของ “char” สามารถเก็บข้อมูลและถูกมองว่าเป็นตัวอักษรได้เพียง 1 ตัวอักษรเท่านั้น

สรุปผลการทดลอง:

จากการทดลองตรวจสอบขนาดข้อมูลและ pointer พบว่าผลลัพธ์การทำงานของโปรแกรมที่เป็นชนิดตัวแปรเก็บค่าต่างๆ สามารถทำงานได้อย่างถูกต้องและตัวแปรที่มี pointer จะจองพื้นที่ตามการทำงานของเครื่องคอมพิวเตอร์ที่เป็น 32-bit หรือ 64-bit

การทดลองที่ 2: ทดลอง Floating Point Error

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main(void) {
```

```
    double x = 0.1 + 0.2;
```

```
    double y = 0.3;
```

```
    printf("x = %.17f\n", x);
```

```
    printf("y = %.17f\n", y);
```

```
    if (x == y) {
```

```
        printf("equal\n");
```

```
    } else {
```

```
        printf("not equal\n");
```

```
    }
```

```
    if (fabs(x - y) < 1e-9) {
```

```
        printf("close enough\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
PS D:\ENGCE124\lab1> ./1-2.exe
```

```
x = 0.30000000000000004
```

```
y = 0.29999999999999999
```

```
not equal
```

```
close enough
```

คำถามท้ายการทดลอง:

1. ทำไม $0.1 + 0.2$ จึงอาจไม่เท่ากับ 0.3

เป็นเพราะคอมพิวเตอร์ทำงานด้วยเลขฐานสอง แต่ค่าข้อมูลเป็นเลขฐานสิบ เมื่อคอมพิวเตอร์นำค่าข้อมูลไปคำนวณจะเกิดเศษส่วนไม่รู้จบและถูกตัดทอนทิ้ง ทำให้ค่าความจริงผิดเพี้ยน

2. ทำไมจึงควรใช้ epsilon ในการเปรียบเทียบ

เพราะค่าประเภท double ในคอมพิวเตอร์มีความคลาดเคลื่อนสะสมจากการแปลงเลขฐานสอง การใช้เครื่องหมาย “==” จะให้ผลลัพธ์เป็นเท็จ ทั้งที่ในทางคณิตศาสตร์มีจะเท่ากัน

3. ถ้าเป็นระบบการเงินควรใช้ floating point หรือไม่ เพราะอะไร

ไม่ควร เพราะค่า floating point ก็มีความคลาดเคลื่อนเช่นเดียวกัน ในการใช้ระบบการเงิน นานๆไปผลลัพธ์การเงินจะเกิดการคลาดเคลื่อนจากเลขเศษที่สะสมเรื่อยๆ

สรุปผลการทดลอง:

ผลทดลอง Floating Point Error พบว่าในค่าข้อมูล float และ double เมื่อนำค่าข้อมูลที่รองรับกับจุดทศนิยมมาคำนวณเพื่อหาความแม่นยำที่ชัดเจน จะได้ผลลัพธ์ที่ใกล้เคียงแทนที่จะเป็นผลลัพธ์ที่ถูกต้อง 100% ซึ่งไม่ควรนำมาใช้กับระบบที่เกี่ยวข้องกับการเงิน

การทดลองที่ 3: ทดลอง UTF-8 ภาษาไทย

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(void) {
```

```
    char text[] = "ก";
```

```
    printf("text = %s\n", text);
```

```
    printf("strlen(text) = %zu bytes\n", strlen(text));
```

```
    for (size_t i = 0; i < strlen(text); i++) {
```

```
        printf("bytes %zu = 0x%02X\n", i, (unsigned char)text[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
PS D:\ENGCE124\lab1> ./1-3.exe
```

```
text = ก ü
```

```
strlen(text) = 3 bytes
```

```
bytes 0 = 0xE0
```

```
bytes 1 = 0xB8
```

```
bytes 2 = 0x81
```

คำถามท้ายการทดลอง:

1. ตัวอักษร ก ใช้กี่ bytes ใน UTF-8
ผลลัพธ์ข้อมูลพบว่า “ก” ใช้พื้นที่ bytes ใน UTF-8
2. strlen() นับจำนวนตัวอักษรหรือจำนวน bytes
strlen() เป็นคำสั่งนับจำนวน bytes
3. เพราะเหตุใด char ch = 'ก'; จึงไม่เหมาะกับการสอน ASCII
เพราะ ASCII ไม่มีรหัสรองรับสำหรับภาษาไทย

สรุปผลการทดลอง:

ผลทดลอง UTF-8 ภาษาไทย พบว่าในภาษาไทยตัวอักษรหนึ่งตัวมีขนาดพื้นที่ที่ไม่สอดคล้องกับจำนวน bytes และรหัส ASCII ไม่สามารถรองรับรหัสภาษาไทย